# Computer Science
# Partial exam rehearsal – Solved version

IGE – 1st Year
Class practice

**Suggested duration:** 1 hour 45 minutes, followed by class discussion.

Try each question before reading its solution. For multiple-choice questions, choose one answer and justify it. Questions and solutions are in English.

**Content.** This rehearsal covers three units. Unit 1 focuses on binary representation, CPU instructions and register states (2 points, 20%). Unit 2 covers Python imports, numeric operations, Boolean conditions, `if-elif-else`, loops and lists (3 points, 30%). Unit 3 covers Arduino programming, error detection, digital inputs and outputs, and an LCD scoreboard (5 points, 50%). These weights are indicative for this academic year and should be used as a guide when preparing for the exam.

# Unit 1 Problems (2.0 points total)

## Q1 - CPU execution

A processor has four registries, labeled R0, R1, R2, and R3 and the following set of operation codes:

1. `0001` Addition
2. `0010` Substraction
3. `1100` Multiplication
4. `1101` Integer Division
5. `1110` Modulo (reminder after Integer division)

The 10-bit instruction is formatted as follows.

`Opcode(4) | Dest(2) | Src1(2) | Src2(2)`

For example, the instruction `1110 00 01 10` means "calculate the reminder after Integer division of the value of registry R1 divided by the value of registry R2 (R1 modulo R2), and store the value in registry R0".

The initial value in the registers are in table 1:

| | |
|---|---|
| R0 | `0b1011` |
| R1 | `0b1000` |
| R2 | `0b0010` |
| R3 | `0b0000` |

Table 1: Initial states

Answer the following questions:

1. Convert to decimal the values in registry (prefixed with `0b`). **(0.5 pts)**

   **Solution:**

   - R0 = 0b1011 = 11
   - R1 = 0b1000 = 8
   - R2 = 0b0010 = 2
   - R3 = 0b0000 = 0

2. Write the instruction to add the values in registries R1 and R2 and store it back in registry R2. This instruction will be instruction 1. **(0.2 pts)**

   **Solution:**

   Opcode for ADD = `0001`
   Dest = R2 = `10`
   Src1 = R1 = `01`
   Src2 = R2 = `10`

   Final instruction:

   `0001 10 01 10`

3. Consider the following instructions:

   - Instruction 2: 0010000001
   - Instruction 3: 1110110100

   Explain what every instruction will do.(0.3 pts)

   **Solution:**

- Instruction 2: `0010 00 00 01` Opcode 0010 = subtraction. R0 = R0 - R1.
- Instruction 3: `1110 11 01 00` Opcode 1110 = modulo. R3 = R1 mod R0.

**(0.3 pts)**

4. Using the initial state, apply instruction1, instruction2 and instruction3. Show the state of the registers after the execution of every instruction. **(1 pts)**

   **Solution:**

   (a) **Instruction 1: ADD R1 and R2 → R2**
   R2 = 8 + 2 = 10 R2 = `0b1010`
   Registers:

   $$R0 = 11\ (\texttt{0b1011})$$
   $$R1 = 8\ (\texttt{0b1000})$$
   $$R2 = 10\ (\texttt{0b1010})$$
   $$R3 = 0\ (\texttt{0b0000})$$

   (b) **Instruction 2: SUB R0 = R0 - R1**
   R0 = 11 - 8 = 3 R0 = `0b0011`
   Registers:

   $$R0 = 3\ (\texttt{0b0011})$$
   $$R1 = 8\ (\texttt{0b1000})$$
   $$R2 = 10\ (\texttt{0b1010})$$
   $$R3 = 0\ (\texttt{0b0000})$$

   (c) **Instruction 3: MOD R3 = R1 mod R0**
   R3 = 8 mod 3 = 2 R3 = `0b0010`
   Final registers:

   $$R0 = 3\ (\texttt{0b0011})$$
   $$R1 = 8\ (\texttt{0b1000})$$
   $$R2 = 10\ (\texttt{0b1010})$$
   $$R3 = 2\ (\texttt{0b0010})$$

# Unit 2 – Python (3 points total)

## Python Q1 – Scenario 4 – Requesting a concise import (0.375 pt)

A financial analyst only needs `sqrt()` from the `math` module and wants to call it without the `math.` prefix. Which option meets the requirement?

A. `import math` followed by `math.sqrt(81)`
B. `import math as sqrt` followed by `sqrt(81)`
C. `from math import sqrt` followed by `sqrt(81)`
D. `from sqrt import math` followed by `sqrt(81)`

**Your choice:**
**Which name becomes available?**

### Solution

**Answer: C.** `from math import sqrt` imports the function under the name `sqrt`, so `sqrt(81)` returns `9.0`. A retains the prefix; B binds the module to `sqrt` and then attempts to call a module; D reverses the module and function names.

**Marking:** correct option, 0.125; justification using the requirement and explaining a distractor or counterexample, 0.25.

## Python Q2 – Scenario 2 – Packing devices (0.375 pt)

A warehouse must pack 137 devices into boxes that hold 12 devices each. The manager needs the number of complete boxes and unpacked devices. Which solution is most appropriate?

A.

```
boxes = 137 / 12
remaining = 137 / 12
```

B.

```
boxes = 137 // 12
remaining = 137 % 12
```

C.

```
boxes = 137 % 12
remaining = 137 // 12
```

D.

```
boxes = int(137 / 12)
remaining = 137 - 12
```

**Your choice and justification:**

### Solution

**Answer: B.** Integer division gives `11` complete boxes and modulo gives `5` remaining devices: `137 = 11 * 12 + 5`. A uses ordinary division; C swaps the results; D subtracts only one box capacity.

**Marking:** correct option, 0.125; justification using the requirement and explaining a distractor or counterexample, 0.25.

## Python Q3 – Scenario 4 – Secure document access (0.375 pt)

A user may open a confidential document when they have permission and their account is not suspended. The following code illustrates how the condition will be used. Replace `<condition>` with one of the options below. Which condition states the requirement most directly?

```
has_permission = True
account_suspended = False

if <condition>:
    print("Access granted")
else:
    print("Access denied")
```

A. `has_permission or not account_suspended`
B. `has_permission and not account_suspended`
C. `not has_permission and account_suspended`
D. `has_permission and account_suspended`

**Your choice and a test that should deny access:**

### Solution

**Answer: B.** Both requirements must hold: `has_permission and not account_suspended`. Access must be denied when permission is `True` and suspension is `True`. It must also be denied when permission is `False`. Option A could grant access without permission. With the sample values, the corrected code prints `Access granted`.

**Marking:** correct option, 0.125; justification using the requirement and explaining a distractor or counterexample, 0.25.

## Python Q4 – B7. Diagnosing an index error (0.375 pt)

An AI assistant generates the following loop and Python raises `IndexError` on the last iteration:

```
for index in range(len(values) + 1):
    print(values[index])
```

Which explanation is most accurate?

A. Lists cannot be used inside `for` loops.

B. `range()` includes its stop value.

C. The `+ 1` creates an index equal to the list length, which is outside the valid range.

D. The loop must begin at index `1`.

### Solution

**Answer: C.** For a list of length `n`, valid indices are `0` to `n - 1`. `range(n + 1)` also produces `n`, causing the error. `range()` excludes its stop value, so B is false. The correct header is `for index in range(len(values)):`.

**Marking:** correct option, 0.125; justification using the requirement and explaining a distractor or counterexample, 0.25.

## Python Q5 – A7. Prioritise outcomes (0.75 pt)

A support request should receive exactly one classification:

1. `"Critical"` when the service is unavailable.
2. `"High"` when data may be lost.
3. `"Medium"` when response time exceeds 5 seconds.
4. `"Normal"` otherwise.

Implement the policy with `if-elif-else`. Create a case where several conditions are true and explain why only one branch runs.

```
service_unavailable = False
possible_data_loss = True
response_time = 8.2

# Implement and print the classification here.
```

**Solution**

```
service_unavailable = False
possible_data_loss = True
response_time = 8.2

if service_unavailable:
    classification = "Critical"
elif possible_data_loss:
    classification = "High"
elif response_time > 5:
    classification = "Medium"
else:
    classification = "Normal"

print(classification)
```

**Output:** `High`.

Both possible data loss and a response time above 5 seconds apply in the supplied case. Python executes only the first matching branch in the `if-elif-else` chain, so the higher-priority classification wins. If `service_unavailable` is changed to `True`, all three conditions hold and the output is `Critical`.

When both Boolean values are `False`, a response time of `8.2` produces `Medium`, while exactly `5` produces `Normal`: the rule says “exceeds”, not “at least”.

**Marking:** correctly ordered `if-elif-else` chain, 0.25; correct conditions, classifications and output, 0.25; a case with several true conditions and explanation of branch priority, 0.25.

## Python Q6 – A5. Preserve a running series (0.75 pt)

A new online service begins with `200` registered users (stored in `initial_users`). A list contains the new users gained each week (`new_users_by_week` stores the new users for each week).

Create `total_users_by_week` so that it contains the initial total followed by the total after every week.

```
initial_users = 200
new_users_by_week = [25, 30, 20, 35]
total_users = initial_users
total_users_by_week = [initial_users]

# Update total_users for every week.
# Preserve every updated total in total_users_by_week.

print(total_users_by_week)
# Expected: [200, 225, 255, 275, 310]
```

### Solution

```
initial_users = 200
new_users_by_week = [25, 30, 20, 35]
total_users = initial_users
total_users_by_week = [initial_users]

for new_users in new_users_by_week:
    total_users = total_users + new_users
    total_users_by_week.append(total_users)

print(total_users_by_week)
```

**Output:** `[200, 225, 255, 275, 310]`.

The running total is `225`, `255`, `275` and `310` after weeks 1–4. Initialise the accumulator before the loop and append inside it to preserve every updated total. The initial `200` remains the first element. With no new weeks, the result is `[200]`.

**Marking:** initial state, 0.25; accumulation, 0.25; storing each total and correct result, 0.25.

# Unit 3 Problems (5 points total)

## Q1 — Error Detection & Correction (1.5)

Consider the following code, it contains several syntax/logic errors.

```
const int led1 = 11
const int led2 = 2;
const int btn = 11;

void setup() {
    pinMode(led1, OUTPUT)
    pinMode(led2, OUTPUT);
    pinMode(btn, OUTPUT);
}

void setup() {
if (digitalRead(btn) = HIGH) {
    pattern = !pattern;
    }
    if (pattern = true) {
        digitalWrite(led1, HIGH);
        digitalWrite(led2, LOW);
    } else {
        digitalWrite(led1, HIGH);
        digitalWrite(led2, HIGH)
        delay(300);
        digitalWrite(led1, LOW);
        digitalWrite(led2, LOW);
    }
    delay(300)
}
```

Solve the following items:

1. Find all 9 errors in the code (0.5 pts).

   **Solution:** There's actually 10 errors in the code, finding 9 gets you all the points:

   (a) Missing semicolon: `const int led1 = 11;`
   (b) Wrong button pin (same as LED1): should not be 11. Fix: `const int btn = 3;` (or another free pin).
   (c) `pinMode(led1, OUTPUT)` missing semicolon.
   (d) `pinMode(btn, OUTPUT);` must be `INPUT`.
   (e) Second function named `setup()`. Must be `loop()`.
   (f) Assignment used instead of comparison: `digitalRead(btn) = HIGH` → `==`.
   (g) Variable `pattern` not declared. Must declare: `bool pattern = false;`.
   (h) Condition `if (pattern = true)` uses assignment. Should be `if (pattern == true)` or simply `if(pattern)`.
   (i) Missing semicolon in `digitalWrite(led2, HIGH)`.
   (j) Missing semicolon in `delay(300)`

2. For every error, explain why it's an error and how to fix it (0.5 pts).

   **Solution:**

   - **Missing semicolons:** C++ uses semicolon to signal when one instruction finishes and the following starts so it's mandatory to add them at the end of every instruction
   - **== instead of =:** In C++ and many other languages, using = will mean assign a value to a variable, in these cases we don't want to change the value of the variable but to check if it has a certain value, therefore we must use double equal.

- **Wrong button pin:** you can't connect 2 devices to the same Arduino Pin
- **INPUT/OUTPUT:** A button is an element from which we are going to read a value, not to write any value in it, therefore we must use INPUT, not OUTPUT whe we set the pin mode.
- **variable pattern**: In order to use a variable, first we have to save space in the computer's memory, doing this is called declaring the variable.

3. Explain what the corrected code is doing (0.5 pts).

   **Solution:** The code reads a pushbutton. When the button is pressed, the Boolean variable `pattern` toggles. When `pattern` is true, the LEDs stay in a fixed on/off pattern, when `pattern` is false, they blink together every 300 ms.

## Q2 - Referee table (1.5)

The following schematic represents your prototype board. The integrated protoboard includes 2 push buttons, labeled as B1, and B2, and two (pull-down resistors). The objective of the prototype is to provide an electronic table to keep track of the score in a football match. Push-button B1 is used to control the score of the local team and B2 is used to control the score of the visiting team (1 push increases the score in 1).

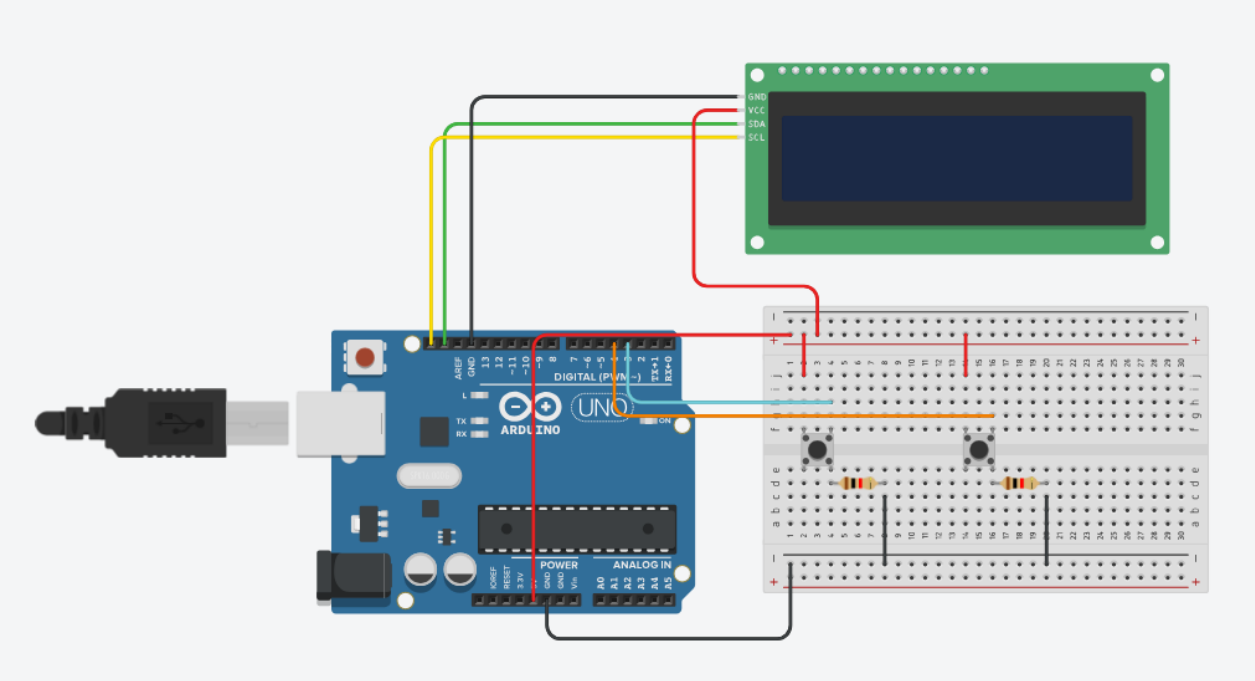



Figure 1: EDEM League electronic table

Based on the provided information and diagram, follow the instructions below to complete the wiring of the prototype.

1. Draw over the diagram the wires needed to connect the circuit (0.5 pts)

   (a) Complete the variables, constants needed. (0.5 pts)

   **Solution:** We need to add constants for the pins that connect the buttons, and variables to keep track of the amount of points that each team has.

```
#include <Adafruit_LiquidCrystal.h>

Adafruit_LiquidCrystal lcd(0);

const int B1 = 2;
const int B2 = 3;

int scoreLocal = 0;
int scoreVisitor = 0;
```

(b) Complete the setup function (1 pts) **Solution:**

```
void setup() {
  lcd.begin(16, 2);

  pinMode(B1, INPUT);
  pinMode(B2, INPUT);

  lcd.print("EDEM league");
}
```

(c) Complete the loop function (1.5 pts)
**Solution:**

```
void loop() {
  bool b1 = digitalRead(B1);
  bool b2 = digitalRead(B2);

  if (b1 == HIGH) {
    scoreLocal += 1;
  }
  if (b2 == HIGH) {
    scoreVisitor += 1;
  }

  lcd.setCursor(0, 1);
  lcd.print(scoreLocal);
  lcd.print(" - "); // clear residual digits
  lcd.print(scoreVisitor);

  delay(100);
}
```

To write the code, you can base it on this example of how an LCD screen works, provided as an example in the next page.

## 0.1 Example code

This code will turn on the LCD screen, write ”hello world” in the first line, and then, in the second line, it writes, every second, the amount of seconds that it has been working. You can use this code as inspiration to write the code of the last exercise.

```
#include <Adafruit_LiquidCrystal.h>
int seconds = 0;
Adafruit_LiquidCrystal lcd_1(0);
void setup()
{
    lcd_1.begin(16, 2);
    lcd_1.print("hello world");
}
void loop()
{
    lcd_1.setCursor(0, 1);
    lcd_1.print(seconds);
    delay(1000); // Wait for 1000 milliseconds
    seconds += 1;
}
```

# Teacher notes and source map

## Preserved source material

Units 1 (CPU execution) and 3 (Arduino) are reproduced verbatim from `main.tex` in this directory. This identifies them as Gabriel Modia's units based on the CPU/Python/Arduino division of the supplied exam. No question variants or additional modification tasks have been added.

## Scoring in the original

Unit 1 totals 2 points, although 0.3 is printed twice for the instruction-decoding item. Unit 3 is headed 5 points, but its Q2 heading says 1.5 while its subitems total 3.5. For this rehearsal, use Unit 1 = 2, Unit 2 = 3, Unit 3 Q1 = 1.5 and Q2 = 3.5, for 10 points overall. Count the Unit 1 decoding item once.

## Arduino solution qualifications

The source Q1 asks for nine errors but its solution lists ten; accept any nine as the source specifies. The corrected loop toggles `pattern` on each iteration where the button reads HIGH, so holding the button can toggle repeatedly. It does not implement one toggle per press.

Similarly, the source Q2 loop increments the score every 100 ms while a button is held. It does not fully implement the stated one-press/one-point requirement. For classroom correction, the implementation below counts a rising edge and assumes ideal, debounced inputs (or a simulator). A real mechanical button also requires debouncing. This is a solution clarification, not an extra student task.

```
// Add these variables to the original global declarations:
bool previousB1 = LOW;
bool previousB2 = LOW;

// Replace the original loop with:
void loop() {
    bool b1 = digitalRead(B1);
    bool b2 = digitalRead(B2);
    if (b1 == HIGH && previousB1 == LOW) {
        scoreLocal += 1;
    }
    if (b2 == HIGH && previousB2 == LOW) {
        scoreVisitor += 1;
    }
    previousB1 = b1;
    previousB2 = b2;
    lcd.setCursor(0, 1);
    lcd.print(scoreLocal);
    lcd.print(" - ");
    lcd.print(scoreVisitor);
    lcd.print("   ");
    delay(10);
}
```

The original `CircuitSolved.png` supplies the wiring answer. The original LCD example is also retained.

## Python question sources

The questions come from the indicated cells. Q3 adds a code template to illustrate the condition, and Q6 explicitly identifies the variables. Q5 uses the prioritisation exercise from Session 06. Original values and answer options are preserved; headings, marks and solutions are added. Cell numbers are one-based and count all notebook cells, including Markdown cells.

**Q1:** Session 04 - Python execution, functions and libraries - practice.ipynb — **Scenario 4 — Requesting a concise import**, cells 30.

**Q2:** Session 05 - Variables, types and numeric operations - practice.ipynb — **Scenario 2 — Packing devices**, cells 27.

**Q3:** Session 06 - Boolean expressions and selection control flow - practice.ipynb — **Scenario 4 — Secure document access**, cells 27.

**Q4:** Session 08 - Loops, lists and visualisation - practice.ipynb — **B7. Diagnosing an index error**, cells 33.

**Q5:** Session 06 - Boolean expressions and selection control flow - practice.ipynb — **A7. Prioritise outcomes**, cells 17, 18.

**Q6:** Session 08 - Loops, lists and visualisation - practice.ipynb — **A5. Preserve a running series**, cells 14, 15.